ResNet Autoencoder: Image Reconstruction with ImageNette Report

Overview

This project explores how a deep learning model can learn to recreate an image after processing it through a neural network. The main goal was to understand how the design of the network affects its ability to preserve important details from an original image.

To investigate this, I used an existing neural network architecture based on ResNet-50, a model commonly used for image recognition. I adapted it into an *autoencoder*, a system that takes an image, extracts information from it, and then attempts to reconstruct the original image using that information.

The project involved four experiments. First, I tested the original model to understand its initial performance. Next, I modified a normalization layer to see whether it affected reconstruction quality. The third experiment changed how the model stored image features, allowing it to preserve more spatial information. Finally, I trained the improved model using a much larger collection of images.

The experiments used the ImageNette dataset, which contains images belonging to ten different categories. I evaluated the models by comparing their reconstructed images with the originals and measuring the differences using mean squared error (MSE).

The final model achieved an MSE of 0.032065, compared with 0.067575 for the original model. This represents an improvement of approximately 52.5% in reconstruction error. Although the reconstructed images still contained visible distortions, the results demonstrated how architectural choices and additional training can improve a neural network's performance.

1. Introduction

Artificial intelligence has become increasingly capable of understanding and processing images. Modern systems can recognize objects, identify patterns, and generate entirely new pictures.

These capabilities are made possible by neural networks, which are computer systems designed to learn patterns from examples.

Instead of following a fixed set of instructions for every possible situation, a neural network adjusts its internal parameters based on the data it receives. Over time, this allows it to learn relationships that would be difficult to describe using traditional programming.

For this midterm project, I wanted to explore a particular type of neural network called an autoencoder.

An autoencoder attempts to recreate the same information it receives. For example, if we give it a photograph of a dog, its objective is to produce an image that looks as close as possible to that original photograph.

This may sound straightforward, but the process is challenging. Before the network can recreate the image, it must transform the original into an internal representation. Some information may be lost during this process.

The challenge is determining which information should be preserved so that the reconstructed image remains recognizable.

I based my experiments on an open-source implementation of a ResNet encoder and decoder developed by Jiahong Chen [1]. This allowed me to examine an existing deep learning architecture, understand how its components work, and make modifications to investigate its performance.

The project focused on two main questions:

1. **Does preserving more information about where objects and features appear in an image improve reconstruction quality?**
2. **Does training the model with more images improve its ability to reconstruct unfamiliar images?**

To answer these questions, I trained several configurations, compared their numerical performance, and examined the images they produced.

2. Understanding the Deep Learning Concepts

Before discussing the experiments, it is helpful to understand the main concepts behind the model.

2.1 What Is a Neural Network?

A neural network is a mathematical model that learns relationships between inputs and outputs.

For image processing, the input is usually a collection of pixel values representing a photograph.

A digital image consists of many small colored points called pixels. In a color image, each pixel generally has three values representing red, green, and blue.

A neural network processes these values through multiple layers. Each layer performs mathematical operations and passes its results to the next layer.

During training, the network adjusts numerical values called *weights*. These weights determine how strongly different pieces of information influence the network's output.

For example, a network trained to recognize dogs might gradually learn patterns associated with fur, ears, eyes, and body shapes.

The important idea is that the network is not explicitly programmed with instructions describing every possible dog. Instead, it learns useful patterns by processing examples.

2.2 What Is a Convolutional Neural Network?

A convolutional neural network, or CNN, is a neural network designed to work particularly well with images.

A CNN examines small regions of an image using mathematical filters. These filters help identify useful visual patterns.

For example, some filters may respond strongly to edges, while others may detect textures, corners, or more complicated structures.

As an image passes through the network, different layers can identify increasingly complex information.

Early layers may recognize simple lines and edges. Later layers can combine these patterns into more meaningful features, such as the shape of an animal or the structure of a building.

CNNs also often reduce the width and height of their internal image representations. This makes processing more efficient and helps the network focus on important features.

However, reducing the size can also remove useful information.

For image classification, losing some detail may not matter. A network might still recognize a dog even if it cannot determine the exact location of every strand of fur.

For image reconstruction, the situation is different. The network must recreate the image, so information about the location and appearance of objects becomes much more important.

2.3 What Is ResNet?

ResNet stands for *Residual Network*. It is a neural network architecture designed to make very deep networks easier to train.

A deep neural network contains many processing layers. Adding more layers can allow a model to learn more complicated patterns, but it also makes training difficult.

One challenge is that useful information and training signals can become harder to propagate through a large number of layers.

ResNet addresses this problem using connections called *residual connections*.

A residual connection provides an additional path that allows information to bypass certain operations and be combined with the output of those operations.

A simple way to understand this is to imagine editing a photograph.

Instead of creating a completely new photograph at every step, we keep the original information and learn which changes should be made to it.

Mathematically, a residual block can be represented as:

Output = F(x) + x

Here, x represents the incoming information, and F(x) represents the transformation learned by the network.

The result combines the original information with the newly learned transformation.

This design helps information and gradients move through deep networks, making them easier to optimize.

For this project, I used ResNet-50, a ResNet architecture with approximately 50 learned layers.

2.4 What Is an Autoencoder?

An autoencoder is a neural network that learns to reconstruct its own input.

It contains two main components:

The encoder receives the original image and transforms it into a representation containing the features that the network has extracted.

The decoder receives that representation and attempts to recreate the original image.

The process can be summarized as:

Original Image → Encoder → Feature Representation → Decoder → Reconstructed Image

For example, imagine giving the encoder a photograph of a church.

The encoder processes the photograph and produces numerical features representing information about the image.

The decoder then uses those features to generate a new image.

The goal is for the reconstructed image to resemble the original church photograph as closely as possible.

The model improves through training by measuring its reconstruction errors and adjusting the decoder to reduce them.

Unlike a traditional image classification task, an autoencoder does not need a label telling it whether an image contains a dog, building, or fish.

The original image itself provides the target that the network is trying to reproduce.

2.5 What Is a Feature Representation?

A feature representation is the information produced by the encoder after it processes an image.

It is not necessarily a normal picture that a person can look at. Instead, it usually consists of many numerical values organized into a structure called a *tensor*.

A tensor can be thought of as a collection of numbers arranged in one or more dimensions.

For images, tensors often include dimensions representing the number of images, feature channels, height, and width.

A feature channel contains numerical responses associated with patterns learned by the network.

One particularly important idea for this project is **spatial information**.

Spatial information describes where features are located relative to one another.

For example, a photograph might contain a dog in the bottom-left corner and a tree in the background.

The model may recognize features associated with both the dog and the tree. However, if it loses information about their locations, reconstructing the original arrangement becomes much more difficult.

This distinction became central to my experiments.

3. The Model Architecture

3.1 The ResNet-50 Encoder

The encoder used in this project was based on ResNet-50.

Rather than training the entire encoder from the beginning, I used weights that had already been learned through ImageNet image classification.

ImageNet is a large collection of labeled images commonly used to train and evaluate computer vision models.

Using pretrained weights means that the encoder already has experience detecting useful visual patterns.

This provides a starting point instead of requiring the network to learn every feature from scratch.

During my experiments, I kept the encoder's weights *frozen*.

Freezing the encoder means its learned parameters were not updated during training. The decoder was the part of the model that learned to improve the reconstructions.

This reduced the amount of training required and allowed me to focus on studying the decoder and the information passed between the two components.

Each input image was resized to 221 × 221 pixels before entering the model.

The original encoder reduced its output to a representation with the dimensions:

2048 × 1 × 1

The number 2048 represents the feature channels produced by ResNet-50.

The dimensions 1 × 1 mean that the features no longer retain a grid of different spatial locations. Instead, information across each feature map has been summarized into a single value.

This happens through a process called *global average pooling*, which averages the values within each feature map.

For recognizing an object, this summary can be useful.

For reconstructing the exact image, however, the summary may discard important information about where visual features originally appeared.

3.2 The Decoder

The decoder is responsible for turning the encoder's numerical representation back into a picture.

It does this through several layers that gradually increase the spatial dimensions of the information being processed.

The decoder uses operations called *transposed convolutions*. These are commonly used in image-generation networks to increase the size of an internal representation.

The decoder also uses residual blocks, similar to those found in ResNet.

Another important component is *max-unpooling*.

During certain pooling operations, the encoder records the locations of the strongest values within small regions. These locations are called pooling indices.

The decoder can use those indices to help position information when increasing the size of its representation.

Therefore, the decoder receives some additional spatial information even when the main encoder output has been reduced to 1 × 1.

The decoder's final output is an image with three color channels and dimensions of 221 × 221 pixels.

I also used a Sigmoid activation function at the end of the decoder.

A Sigmoid function restricts output values to a range between 0 and 1.

This is useful because the image targets in the experiment were represented using pixel values in that same range.

3.3 Preserving More Spatial Information

The most important architectural change in this project involved how the encoder represented the image before sending information to the decoder.

The original model used global average pooling to produce:

2048 × 1 × 1 = 2,048 values

I modified the encoder to preserve its feature maps before global average pooling, producing:

2048 × 7 × 7 = 100,352 values

The difference is significant.

Instead of summarizing each feature channel into a single number, the modified model retained a grid containing 49 spatial positions for each channel.

This allowed the representation to retain more information about the arrangement of features.

For example, if an image contains a dog on one side and a tree on the other, preserving a spatial grid can help the decoder distinguish their approximate locations.

The modified representation contains 49 times as many numerical values as the original representation.

This is important because the modified model has access to substantially more information. Any improvement cannot automatically be attributed only to the preservation of spatial positions.

The experiment instead investigates how this larger, spatially organized representation performs in practice.

4. Dataset and Experimental Setup

4.1 The ImageNette Dataset

To train and evaluate the model, I used ImageNette.

ImageNette is a smaller dataset derived from ImageNet. It contains photographs belonging to ten image categories, making it useful for experimenting with image classification and computer vision models.

The downloaded dataset contained:

- **9,469 training images**
- **3,925 validation images**
- **10 image categories**

Examples include different dog breeds, fish, churches, and parachutes.

The variety of images was useful because the model needed to reconstruct different shapes, colors, textures, and backgrounds.

I used PyTorch, a deep learning framework for Python, to load and process the images.

PyTorch provides tools for creating neural networks, training their parameters, and performing calculations efficiently on graphics processing units.

4.2 Preparing the Images

Neural networks require inputs in a consistent numerical format.

Before training, each image was resized to 221 × 221 pixels.

The images were then converted into tensors containing red, green, and blue pixel values.

These values were scaled to the range from 0 to 1.

Before passing the images through the pretrained encoder, I also applied normalization using the means and standard deviations associated with ImageNet.

Normalization adjusts the distribution of the input values to match what the pretrained encoder expects.

This matters because the ResNet-50 encoder was originally trained using similarly normalized images.

The decoder was trained to reproduce the original images in the 0-to-1 pixel range.

4.3 Dividing the Dataset

For the initial experiments, I used a smaller randomly sampled portion of ImageNette to reduce training time.

The initial data included 512 training images and 128 validation images.

The training images were used to update the decoder's parameters.

The validation images were used to monitor how well the model reconstructed images that were not part of its training subset.

The remaining 3,797 images from ImageNette's validation partition were used for larger-scale evaluation.

This separation helped measure reconstruction performance beyond the small group of images used during training.

For the final experiment, I used all 9,469 images in the training partition, while continuing to use the 128-image validation subset to select checkpoints.

4.4 Hardware and Software

I performed the experiments locally on a computer equipped with an NVIDIA GeForce RTX 5070 Ti graphics card with approximately 16 GB of GPU memory.

I used Python 3.11 and a CUDA-enabled version of PyTorch.

A graphics processing unit, or GPU, is particularly useful for deep learning because neural networks perform many mathematical operations simultaneously.

Using GPU acceleration made it practical to train and evaluate the ResNet-based autoencoder on thousands of images.

5. How the Model Was Trained

Training an autoencoder involves repeatedly asking the model to reconstruct images, measuring how different the results are from the originals, and adjusting its parameters.

The process follows several steps.

First, the model receives a batch of training images.

The encoder processes those images and produces their feature representations.

The decoder then attempts to reconstruct the original images using those features.

Next, the reconstructed images are compared with the originals using a mathematical measure called a *loss function*.

A loss function provides a number describing how far the model's output is from the desired result.

For this project, I used **mean squared error (MSE)**.

5.1 Understanding Mean Squared Error

MSE measures the average squared difference between the original image's pixel values and the reconstructed image's pixel values.

The calculation follows three basic steps:

1. Compare each reconstructed pixel value with the corresponding original pixel value.
2. Square the differences so that positive and negative errors do not cancel each other.
3. Calculate the average of those squared differences.

For example, suppose the original value for a particular color channel is 0.8 and the reconstructed value is 0.6.

The difference is 0.2, and its squared error is 0.04.

The model performs this comparison across all the pixel values in the images.

A lower MSE means the reconstructed image is numerically closer to the original.

An MSE of zero would indicate that the compared pixel values match exactly.

However, MSE does not perfectly describe how natural an image looks to a person.

A reconstructed image might achieve a relatively low MSE while still appearing blurry, noisy, or distorted.

This became an important observation in the experiments.

5.2 Improving the Model Through Training

After calculating the loss, the network uses a process called *backpropagation* to determine how its trainable parameters should change.

Backpropagation calculates how different parameters influence the reconstruction error.

An optimization algorithm then uses this information to update the parameters.

For my experiments, I used the Adam optimizer.

Adam is an optimization method that automatically adjusts parameter updates using information from previous gradients.

The *learning rate* controls the general size of these updates.

I initially used a learning rate of 0.0003, later reducing it to 0.0001 for additional experiments.

The smaller training experiments used a batch size of four images.

A batch is simply a group of images processed together before the model updates its parameters.

For the final experiment, I increased the batch size to eight.

The model was trained over multiple *epochs*. One epoch represents one complete pass through the selected training dataset.

After each epoch, I measured training and validation loss.

Whenever the validation loss improved, I saved the model's parameters in a file called a checkpoint.

This allowed me to retain the best-performing version observed during training instead of automatically using the model from the final epoch.

Throughout the experiments, the encoder remained frozen and in evaluation mode so that its pretrained behavior remained consistent.

6. Experiments and Observations

I conducted four main experiments to understand which changes improved image reconstruction.

6.1 Experiment 1: Testing the Original Model

The first experiment established a starting point for comparison.

I used the original ResNet-based autoencoder with its 2048 × 1 × 1 feature representation.

The decoder was trained using 512 images for three epochs.

As training progressed, the reconstruction error decreased, indicating that the decoder was learning to reproduce some information from the input images.

The best validation MSE was approximately **0.0732**.

However, the reconstructed images looked noticeably different from the originals.

Most outputs appeared gray, with strong repetitive patterns and very little accurate color.

Some general object outlines were visible, but the images lacked fine details and natural textures.

For example, the network could sometimes produce faint shapes corresponding to the original object without accurately reconstructing its appearance.

This demonstrated an important limitation of the model.

Even when a neural network improves its numerical reconstruction score, the output may still be difficult for a person to recognize.

The original experiment provided a baseline against which I could evaluate later changes.

Figure 1. Original ResNet-50 Autoencoder Reconstructions



The figure compares four original images with their reconstructed versions. The reconstructions contain recognizable outlines in some cases but substantial noise, poor color accuracy, and missing details.

6.2 Experiment 2: Testing the Effect of Batch Normalization

The second experiment investigated whether a normalization operation near the end of the decoder was affecting the reconstructed images.

The original decoder contained a layer called *Batch Normalization*, or BatchNorm.

BatchNorm is commonly used in neural networks to adjust the values produced by a layer. This can help stabilize training and make optimization easier.

However, normalization near an image-generation output can also influence the range and distribution of the values used to construct the image.

I wanted to investigate whether removing the final BatchNorm layer would improve reconstruction quality.

To test this, I created a modified version of the decoder where the final BatchNorm layer was replaced with an identity operation.

An identity operation simply passes information through without changing it.

Immediately after making the change, validation MSE increased from approximately 0.073226 to 0.076782.

This meant the modification initially made the reconstruction error worse.

After additional training, the modified decoder improved to an MSE of approximately **0.066912**.

However, additional training itself can improve performance.

To make the comparison more meaningful, I also trained the original decoder for two additional epochs.

After this training, the original decoder achieved a validation MSE of approximately **0.067150**.

The final results were very close:

Decoder configuration	**Validation MSE**
Original decoder with BatchNorm	0.067150
Modified decoder without final BatchNorm	0.066912

The modified version achieved a slightly lower error, but the difference was small.

The images produced by both versions also remained heavily distorted.

Therefore, I concluded that removing the final BatchNorm layer did not provide a substantial reconstruction advantage in this experiment.

Because the training histories were not perfectly identical, this comparison should be considered exploratory rather than a definitive measurement of BatchNorm's effect.

Figure 2. Reconstruction Comparison With and Without Final BatchNorm



The examples show that changing the normalization layer produced only modest differences in the reconstructed images.

6.3 Experiment 3: Preserving Spatial Features

The third experiment involved the most important architectural change.

I suspected that the original model was losing too much information when it reduced the encoder's output to 1 × 1.

Although this representation contains many feature channels, it does not preserve a grid showing where those features were detected.

To investigate this, I removed the global average pooling operation from the encoder.

The modified encoder produced a 2048 × 7 × 7 representation instead of 2048 × 1 × 1.

This preserved 49 spatial positions for each feature channel.

The idea was that retaining more information about the location of visual features would make the decoder's reconstruction task easier.

I started with decoder weights learned during the previous experiments and continued training the modified model on the 512-image training subset.

Before the additional training, validation MSE was approximately 0.077671.

After five additional epochs, the best validation MSE decreased to approximately **0.047845**.

When evaluated on the larger 3,797-image evaluation set, the spatial model achieved an MSE of **0.048511**, compared with 0.067575 for the original configuration.

The improvement was visible in the images.

Objects became easier to distinguish, and the model reconstructed more of the original image structure.

For example, the outline of a dog, the architecture of a church, and the arrangement of objects within a scene became more recognizable.

However, the reconstructed images still contained substantial grid-like patterns, incorrect colors, and missing details.

The experiment suggested that preserving a larger spatial feature representation can help image reconstruction.

It did not prove that spatial preservation was the only cause of the improvement, because the modified representation contained more information and the model received additional training.

6.4 Experiment 4: Training With the Full Dataset

The final experiment investigated whether exposing the spatial model to more training images would improve its reconstructions.

Until this point, most training had used only 512 images.

Although this was sufficient to test the architecture, it represented a small portion of the available ImageNette dataset.

I therefore continued training the spatial model using all **9,469 training images**.

I started from the previously trained spatial decoder checkpoint.

The batch size was increased to eight, and the model received three additional epochs of training.

The training MSE decreased from approximately 0.0441 during the first epoch to 0.0335 during the third epoch.

Validation MSE also improved, decreasing from approximately 0.0378 to 0.0330.

After training, I evaluated the saved model on the 3,797-image evaluation set.

The final model achieved an MSE of **0.032065**, the lowest error among the configurations tested.

The reconstructed images also showed noticeable improvements.

More color information became visible, and the shapes of objects were generally easier to identify.

For example, the dog reconstruction displayed more of the original white and brown coloring, the church reconstruction preserved more architectural detail, and the parachute scene recovered more of the sky's blue color.

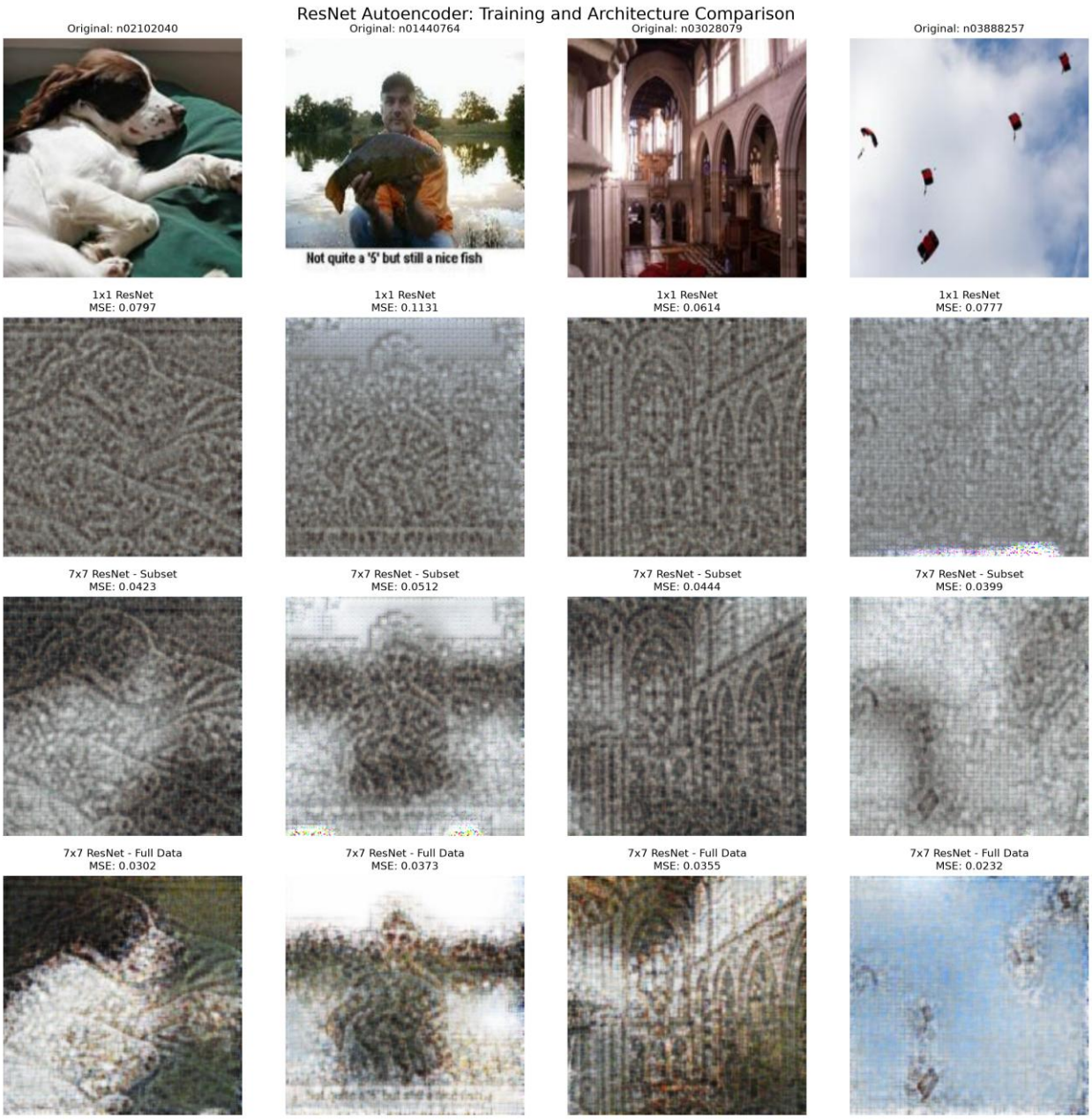
Nevertheless, the images were still far from perfect.

The outputs continued to contain repetitive grid patterns, blurred regions, and missing fine details.

The final experiment demonstrated that additional training on a larger dataset improved the performance of the spatial model.

However, it also involved more optimization steps, so the improvement cannot be attributed exclusively to dataset size.

Figure 3. Comparison of Original, Spatial, and Full-Dataset Reconstructions



The figure compares identical images across the original 1 × 1 model, the spatial 7 × 7 model trained on a subset, and the spatial model trained on the full dataset.

7. Results and Evaluation

To evaluate the experiments, I used both numerical measurements and visual comparisons.

Numerical measurements helped determine whether the reconstructed pixel values became closer to the original images.

Visual comparisons helped assess whether the reconstructed images actually looked more recognizable.

7.1 Quantitative Results

The main evaluation metric was mean squared error.

I also used *peak signal-to-noise ratio*, or PSNR.

PSNR is another way of measuring differences between an original image and a reconstructed image.

It is calculated from the reconstruction error and expressed in decibels (dB).

For this project, a higher PSNR indicates a reconstruction with less numerical distortion.

While MSE is better when it is lower, PSNR is better when it is higher.

The results were:

Model configuration	**Evaluation MSE**	**PSNR**
Constant-gray baseline	0.083396	10.79 dB
Original ResNet, 1 × 1	0.067575	11.70 dB
Spatial ResNet, 7 × 7, small dataset	0.048511	13.14 dB
Spatial ResNet, 7 × 7, full dataset	**0.032065**	**14.94 dB**

The constant-gray baseline represents a simple comparison method that generates a uniform gray output rather than reconstructing the image's actual contents.

Including this baseline helps determine whether the neural network performs better than a simple prediction.

The original ResNet model performed better than the constant-gray baseline.

The spatial model improved further.

Finally, training the spatial model on the complete training dataset produced the best numerical performance.

Compared with the original 1 × 1 model, the final configuration reduced MSE by approximately **52.5%**.

Compared with the spatial model trained on the smaller dataset, full-dataset training reduced MSE by approximately **33.9%**.

PSNR also increased from 11.70 dB for the original model to 14.94 dB for the final model.

These measurements show a consistent improvement in numerical reconstruction performance.

7.2 Visual Evaluation

Numerical results alone do not tell the complete story.

A model can produce a lower MSE while still generating images that look unnatural or distorted.

For this reason, I also compared the reconstructions visually.

The original model often produced gray images with faint outlines and strong noise patterns.

The spatial model made several objects more recognizable, particularly when their overall shapes were distinct.

The full-dataset model recovered additional color and improved some structural details.

However, none of the configurations produced consistently realistic reconstructions.

Many outputs contained repetitive lines and grid-like artifacts.

These distortions sometimes made objects look as though they were covered by a mesh or textured pattern.

This suggests that, while the network learned useful information about the original images, it struggled to reproduce natural textures and fine visual details.

The visual results are important because they prevent an overly optimistic interpretation of the numerical improvements.

The final model is better than the original configuration according to the measured reconstruction error, but its images are still noticeably different from the originals.

8. Discussion and Limitations

The experiments provided several useful insights into how autoencoder design and training affect image reconstruction.

8.1 Why Spatial Information Matters

One of the clearest observations was the improvement associated with retaining the 7 × 7 feature representation.

The original encoder was designed primarily for image recognition, where summarizing an image into a compact representation can be useful.

For example, a classification network only needs enough information to determine whether an image contains a dog.

It does not necessarily need to remember the precise location of every feature in the picture.

Image reconstruction requires more detailed information.

The decoder must determine not only what visual features are present, but also approximately where they belong.

Preserving the 7 × 7 feature map allowed more spatially organized information to reach the decoder.

The resulting improvement supports the idea that spatial representations are important for image reconstruction.

However, the experiment did not isolate spatial preservation as the only variable.

The modified representation contained substantially more numerical values, and its decoder received additional training.

A more controlled experiment would train both architectures using identical data, training durations, and comparable initialization conditions.

This would help separate the effect of architecture from the effect of additional training.

8.2 Why More Training Data Helped

The full-dataset experiment produced another substantial improvement.

Training with 9,469 images exposed the model to more examples of different objects, backgrounds, lighting conditions, textures, and colors.

With more examples, the decoder had additional opportunities to learn how different feature representations relate to original image appearances.

However, using more images also resulted in more training updates.

Therefore, the improvement may reflect both the additional variety of images and the additional optimization performed.

A future experiment could compare different dataset sizes while holding the number of training updates constant.

8.3 Why the Reconstructions Still Looked Distorted

Despite the improvements, the final images continued to contain noticeable visual artifacts.

One possible explanation is the decoder's use of transposed convolution layers.

These layers increase the spatial dimensions of the feature maps, but certain configurations can produce uneven patterns in the reconstructed images.

Such patterns are sometimes called *checkerboard artifacts*.

The repeated grid-like textures in my results are consistent with this type of problem.

However, the experiments did not directly determine whether transposed convolution was the only cause.

A future improvement would be to test alternative methods of increasing image size.

For example, the decoder could first enlarge a feature map using interpolation and then apply a regular convolution.

This might reduce some of the repetitive artifacts.

Another possible improvement would be to use reconstruction losses that account for perceptual image quality rather than relying exclusively on pixel-level MSE.

8.4 Limitations of the Pretrained Encoder

Another limitation was that the encoder remained frozen throughout training.

This was useful because it reduced computational requirements and made the experiments easier to manage.

However, the pretrained ResNet-50 encoder was originally designed to support image classification.

The features that are useful for identifying an object are not necessarily ideal for reconstructing every detail of that object.

A future experiment could allow some of the encoder's layers to be updated during training.

This process is called *fine-tuning*.

Fine-tuning would allow the encoder to adapt its representations to the reconstruction task instead of relying entirely on features learned for classification.

8.5 Limitations of the Experimental Comparisons

The experiments were designed to investigate how different modifications affected reconstruction behavior.

However, they were not all perfectly controlled comparisons.

Some models received additional training epochs, and later configurations inherited decoder weights from earlier experiments.

These differences can influence performance independently of architecture.

The evaluation images were also inspected during the broader development process, so the evaluation set should not be described as a completely untouched final test set.

A stronger future study would use matched training conditions, multiple random seeds, and a separate test set that remained unused until all model choices were finalized.

These limitations do not remove the measured improvements, but they are important when interpreting their causes.

9. Conclusion

This project investigated how a ResNet-50-based autoencoder reconstructs images and how changes to its architecture and training procedure affect performance.

I began with an existing encoder-decoder implementation and trained its decoder to recreate images from the ImageNette dataset.

The original architecture used a compact 2048 × 1 × 1 feature representation. Although the model learned to reproduce some outlines and basic structures, the reconstructed images were heavily distorted.

I then investigated whether removing the final BatchNorm layer would improve the results. The modified decoder achieved a slightly lower validation error after additional training, but the difference was small.

The most important change was preserving a 2048 × 7 × 7 spatial feature representation instead of reducing the features to a single spatial position.

This configuration performed substantially better in the experiments and produced more recognizable reconstructions.

I then continued training the spatial model using the complete ImageNette training dataset.

The final model achieved an evaluation MSE of **0.032065** and a PSNR of **14.94 dB**.

This represented approximately a 52.5% reduction in MSE compared with the initial 1 × 1 configuration.

Although the numerical improvement was substantial, the reconstructed images continued to show grid-like artifacts, blurred regions, and missing details.

The project helped demonstrate an important principle in deep learning: a model's architecture determines what information it can retain, while its training procedure influences how effectively it uses that information.

It also showed why evaluating a neural network requires more than examining a single performance number.

By comparing both numerical metrics and reconstructed images, I was able to identify improvements while recognizing the limitations of the final model.

Overall, the experiments provided practical experience with convolutional neural networks, pretrained models, autoencoders, reconstruction loss, optimization, and evaluating deep learning systems.